Step 3 — The Test-Coverage Reviewer, the Supervisor (Routing + Compile) — Solution

> Back to the step · Hint 1 · Hint 2

Story: this finishes the team — adding the Test-Coverage reviewer, and a Supervisor that routes by what kind of diff it's actually looking at, and compiles every reviewer's findings into one clean, unified review. **If not:** three separately-correct reviewers stuck together with no coordination produces three separate reports, not one genuinely useful review — and the whole point of going multi-agent never actually shows up.

The Test-Coverage reviewer

```
# src/agents/coverage_reviewer.py
# Flag a changed code file that has no accompanying test change in the same
# diff. A deterministic heuristic - cheap and good enough for a first pass.
from src.schemas import Finding


def is_code(path):
    return path.endswith((".py", ".js", ".ts"))


def is_test(path):
    return "test" in path.lower()


def run_coverage(files):
    changed_paths = []
    for one_file in files:
        changed_paths.append(one_file["path"])
    test_changed = False
    for path in changed_paths:
        if is_test(path):
            test_changed = True
    findings = []
    for one_file in files:
        path = one_file["path"]
        if is_code(path) and not is_test(path) and not test_changed:
```

```
            findings.append(Finding(
                severity="medium", category="test-coverage",
                message=f"{path}: changed code with no test changes in this PR",
            ))
    return findings
```

The Supervisor: routing and compiling

```
# src/agents/supervisor.py
# choose_reviewers: route by which file categories changed. decide_next: the
# next reviewer that still needs to run, else compile. compile_review: merge
# findings into one de-duplicated, severity-sorted comment. The Supervisor
# never does a review itself -- it only routes and compiles.
from langgraph.types import Command

from src.diff import categorize
from src.utils.logger import get_logger

SEVERITY_ORDER = {"high": 0, "medium": 1, "low": 2}

def choose_reviewers(files):
    categories = set()
    for one_file in files:
        categories.add(categorize(one_file["path"]))
    reviewers = []
    if "code" in categories:
        reviewers.append("security")
        reviewers.append("coverage")
    # Style applies to code and docs, but not a config-only change.
    if categories - {"config"}:
        reviewers.append("style")
    return reviewers

def decide_next(state):
    for reviewer in state["reviewers"]:
        if reviewer not in state["done"]:
            return reviewer
    return "compile"

def compile_review(findings):
    seen = set()
    unique = []
    for finding in findings:
        key = (finding.category, finding.message)
        if key not in seen:
```

```
            seen.add(key)
            unique.append(finding)
    def severity_rank(finding):
        return SEVERITY_ORDER.get(finding.severity, 3)

    unique.sort(key=severity_rank)
    if not unique:
        return "CodeGuard: no issues found."
    lines = ["CodeGuard review:"]
    for finding in unique:
        lines.append(f"  [{finding.severity.upper()}] "
                     f"({finding.category}) {finding.message}")
    return "\n".join(lines)

def supervisor(state):
    logger = get_logger(__name__)
    target = decide_next(state)
    logger.info(f"supervisor -> {target}")
    return Command(goto=target,
                   update={"routing_log": state["routing_log"] + [target]})
```

The state and the graph

```
# src/state.py
from typing import TypedDict

from src.agents.supervisor import choose_reviewers
from src.diff import parse_diff


class ReviewState(TypedDict):
    diff: str
    files: list
    reviewers: list
    done: list
    findings: list
    review: str
    routing_log: list


def start_state(diff):
    files = parse_diff(diff)
    return {
        "diff": diff,
        "files": files,
```

```
        "reviewers": choose_reviewers(files),
        "done": [],
        "findings": [],
        "review": "",
        "routing_log": [],
    }
```

```
# src/graph.py
# START -> supervisor; the supervisor routes to each pending reviewer (which
# runs and returns to the supervisor), then to compile -> END. The reviewers
# that run depend on choose_reviewers(files), so a docs-only diff never calls
# Security or Coverage.
from langgraph.graph import END, START, StateGraph
from langgraph.types import Command

from src.agents.coverage_reviewer import run_coverage
from src.agents.security_reviewer import run_security
from src.agents.style_reviewer import run_style
from src.agents.supervisor import compile_review, supervisor
from src.state import ReviewState


def build_graph(model, style_collection):
    def security_node(state):
        findings = run_security(state["files"])
        return Command(goto="supervisor", update={
            "findings": state["findings"] + findings,
            "done": state["done"] + ["security"],
        })

    def coverage_node(state):
        findings = run_coverage(state["files"])
        return Command(goto="supervisor", update={
            "findings": state["findings"] + findings,
            "done": state["done"] + ["coverage"],
        })

    def style_node(state):
        findings = run_style(model, style_collection, state["files"])
        return Command(goto="supervisor", update={
            "findings": state["findings"] + findings,
            "done": state["done"] + ["style"],
        })

    def compile_node(state):
        return Command(goto=END,
```

```
                        update={"review": compile_review(state["findings"])})

    graph = StateGraph(ReviewState)
    graph.add_node("supervisor", supervisor)
    graph.add_node("security", security_node)
    graph.add_node("coverage", coverage_node)
    graph.add_node("style", style_node)
    graph.add_node("compile", compile_node)
    graph.add_edge(START, "supervisor")
    return graph.compile()
```

`main.py` now builds the style store and the graph and runs a diff:

```
# main.py  (Step 3 version)
from src.agents.supervisor import compile_review  # noqa: F401
from src.config import load_config
from src.graph import build_graph
from src.models.llm import get_model
from src.rag.ingest import build_style_store
from src.rag.store import get_collection
from src.state import start_state

DIFF = """--- a/app.py
+++ b/app.py
+api_key = "SUPERsecret12345"
+def getUserName(id):
+    return eval(id)
"""


def main():
    config = load_config()
    model = get_model(config)
    collection = build_style_store(get_collection())
    app = build_graph(model, collection)
    final = app.invoke(start_state(DIFF))
    print("Routing:", final["routing_log"])
    print(final["review"])


if __name__ == "__main__":
    main()
```

The tests

```
# tests/unit/test_routing.py
# Run from the repo root: python -m tests.unit.test_routing
from src.agents.supervisor import choose_reviewers
from src.diff import parse_diff

CODE = "--- a/app.py\n+++ b/app.py\n+x = 1\n"
DOCS = "--- a/README.md\n+++ b/README.md\n+docs line\n"
CONFIG = "--- a/config.yaml\n+++ b/config.yaml\n+key: value\n"


def test_code_runs_all():
    reviewers = set(choose_reviewers(parse_diff(CODE)))
    result = "OK"
    if reviewers != {"security", "coverage", "style"}:
        result = "WRONG"
    print(f"case 1 (code -> all three)    -> {result}")


def test_docs_only_style():
    reviewers = choose_reviewers(parse_diff(DOCS))
    result = "OK"
    if reviewers != ["style"]:
        result = "WRONG"
    print(f"case 2 (docs-only -> style)   -> {result}")


def test_config_skips_style():
    reviewers = choose_reviewers(parse_diff(CONFIG))
    result = "OK"
    if "style" in reviewers:
        result = "WRONG"
    print(f"case 3 (config -> no style)   -> {result}")


if __name__ == "__main__":
    test_code_runs_all()
    test_docs_only_style()
    test_config_skips_style()
```

```
# tests/unit/test_compile.py
# Run from the repo root: python -m tests.unit.test_compile
from src.agents.supervisor import compile_review
from src.schemas import Finding


def test_dedup_and_sort():
```

```
    findings = [
        Finding(severity="medium", category="security", message="x: eval"),
        Finding(severity="high", category="security", message="x: secret"),
        Finding(severity="medium", category="security", message="x: eval"),
    ]
    out = compile_review(findings)
    result = "OK"
    if out.count("eval") != 1:
        result = "WRONG"
    if out.index("HIGH") > out.index("MEDIUM"):
        result = "WRONG"
    print(f"case 1 (dedup + severity sort) -> {result}")


def test_empty_is_clean():
    out = compile_review([])
    result = "OK"
    if "no issues found" not in out:
        result = "WRONG"
    print(f"case 2 (no findings -> clean)  -> {result}")


if __name__ == "__main__":
    test_dedup_and_sort()
    test_empty_is_clean()
```

```
# tests/integration/test_live_review.py
# Real model + RAG. Run from the repo root:
#   python -m tests.integration.test_live_review
from src.config import load_config
from src.graph import build_graph
from src.models.llm import get_model
from src.rag.ingest import build_style_store
from src.rag.store import get_collection
from src.state import start_state

DIRTY = """--- a/app.py
+++ b/app.py
+api_key = "SUPERsecret12345"
+def getUserName(id):
+    return eval(id)
"""
DOCS = "--- a/README.md\n+++ b/README.md\n+a new documentation line\n"


def build_app():
```

```
    config = load_config()
    model = get_model(config)
    collection = build_style_store(get_collection())
    return build_graph(model, collection)


def test_full_review():
    final = build_app().invoke(start_state(DIRTY))
    review = final["review"].lower()
    ok = "secret" in review and "snake_case" in review and "test" in review
    result1 = "WRONG"
    if ok:
        result1 = "OK"
    print(f"case 1 (full review, all 3)    -> {result1}")


def test_docs_skips_security():
    final = build_app().invoke(start_state(DOCS))
    ok = "security" not in final["routing_log"]
    result2 = "WRONG"
    if ok:
        result2 = "OK"
    print(f"case 2 (docs-only skips security) -> {result2}")


if __name__ == "__main__":
    test_full_review()
    test_docs_skips_security()
```

Expected unit output:

```
case 1 (code -> all three)    -> OK
case 2 (docs-only -> style)   -> OK
case 3 (config -> no style)   -> OK
```

```
case 1 (dedup + severity sort) -> OK
case 2 (no findings -> clean)  -> OK
```

The multi-agent value is in these two files: **routing** (a docs-only diff never pays for a security check) and **compiling** (one de-duplicated, severity-sorted comment instead of three raw reports). That's the whole thing this project exists to prove generalizes.